

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::attachment Struct Reference	17
8.1.1	Detailed Description	17
8.2	untangle::async::execution< actionT > Class Template Reference	17
8.2.1	Detailed Description	19
8.2.2	Constructor & Destructor Documentation	19
8.2.2.1	execution() [1/2]	19
8.2.2.2	execution() [2/2]	19
8.2.2.3	~execution()	20
8.2.3	Member Function Documentation	20
8.2.3.1	create_instance()	20
8.2.3.2	bind_action_and_method()	21
8.2.3.3	bind_action_and_function()	21
8.2.3.4	is_running()	22
8.2.3.5	is_busy()	22
8.2.3.6	run()	22
8.2.3.7	start()	23
8.2.3.8	stop()	23
8.2.3.9	add_action()	23
8.2.3.10	attach()	24
8.2.3.11	detach()	24
8.2.3.12	results()	25
8.2.4	Member Data Documentation	25
8.2.4.1	action_execute	25
8.2.4.2	action_stop	26

8.2.4.3 <code>action_is_running</code>	26
8.2.4.4 <code>on_finished</code>	26
8.3 <code>untangle::async::execution_poll</code> Class Reference	26
8.3.1 Detailed Description	27
8.3.2 Member Function Documentation	27
8.3.2.1 <code>add()</code>	27
8.3.2.2 <code>remove()</code>	27
8.3.2.3 <code>is_running()</code>	28
8.3.2.4 <code>get()</code>	28
8.4 <code>untangle::async::invalid_attachment</code> Struct Reference	28
8.4.1 Detailed Description	28
8.4.2 Constructor & Destructor Documentation	28
8.4.2.1 <code>invalid_attachment()</code>	28
8.4.3 Member Function Documentation	29
8.4.3.1 <code>what()</code>	29
9 File Documentation	31
9.1 <code>async.hpp</code>	31
Index	39

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a list of callables that are invoked on other thread. The callables are pumped into the list by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and `MSVC` releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("oneoff");
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
action();
// run the execution
execution->run();

// add the execution object to the running poll
untangle::async::execution_poll::get().add(*execution);

// wait the polled executions to finish
while(untangle::async::execution_poll::get().is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

```

```

}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

building the tests

```

cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build

```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access. The `actuator` submodule has to be present: `git submodule update --init`.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed `std::mutex` throws `std::system_error` from inside `libsystem` before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	17
untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Single-tone poll that reports whether any execution added to it is still running	26
untangle::async::invalid_attachment	
Raised by execution::attach() when the attachment asked for is refused	28

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		31
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

At namespace scope, so it is one type rather than one per specialisation: an execution may be attached by an execution of any specialisation, and the chain is walked without knowing any link's action type. It dies with its execution, so it doubles as a liveness token - a link that still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- `execution ()`
Constructs an execution with the default name. Call `create_instance()` instead.
- `execution (std::string exec_name)`
Constructs a named execution. Use `create_instance()` for one that is to be bound.
- `~execution ()`
Destroys the execution, once its worker has left.
- `bool is_running () const`
Is this execution still working?
- `bool is_busy () const`
Is this execution working through its action list?
- `void run ()`
Runs what is queued on a worker of its own, once, and keeps the results.
- `void start ()`
Starts a continuous worker that runs whatever is queued until `stop()`.
- `void stop ()`
Stops the worker, after the actions already queued have run.
- `template<typename... Args>`
`bool add_action (actionT action, Args... args)`
Queues an action, bound to args, and says whether it was taken.
- `template<typename otherT>`
`void attach (const std::shared_ptr< otherT > &other)`
Attaches another execution, so that this one triggers it.
- `template<typename otherT>`
`bool detach (otherT &other)`
Detaches an execution previously attached by `attach()`.
- `auto results () const`
The return values of the actions run by `run()`, in the order they ran.

Static Public Member Functions

- `template<typename... Args>`
`static std::shared_ptr< execution > create_instance (Args &&... args)`
Creates an execution owned by a `std::shared_ptr`, which is what binding to it requires.
- `template<typename classT, typename T>`
`static void bind_action_and_method (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)`
Binds asynchronously an external action to a class function member.
- `template<typename T>`
`static void bind_action_and_function (actionT &action, const T &Fn, const std::shared_ptr< execution > &async_exec)`
Binds asynchronously an external action to a plain function.

Public Attributes

- `std::function< void(void)> action_execute`
Connection point that runs this execution's queued actions. Bound to `execute_actions()`.
- `std::function< void(void)> action_stop`
Connection point that stops this execution. Bound to `stop()`.
- `std::function< bool(void)> action_is_running`
Connection point that reports whether this execution is running. Bound to `is_running()`.
- `std::function< void(void)> on_finished`
Called on the worker's thread once the action list has drained. Assigned by the caller.

Static Public Attributes

- `static constexpr auto default_name = "default_name"`
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

An async execution provides a mechanism to queue actions and execute them sequentially on a separate thread. It may also attach another [execution](#) object and trigger its actions. This way actions of different types may be executed on the same thread.

The mechanism relies on an "asynchronous binding" created by [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#).

Template Parameters

<code>actionT</code>	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
----------------------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 `execution()` [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs an execution with the default name. Call [create_instance\(\)](#) instead.

The name identifies this execution in the warnings it reports, and is otherwise unused.

8.2.2.2 `execution()` [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a named execution. Use [create_instance\(\)](#) for one that is to be bound.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution, once its worker has left.

The worker is detached and cannot be joined, so this waits on the "running" state instead. It detaches this execution from whatever attached it first, and returns at once for one that was never started.

Attention

Safe only because running_ is cleared as the very last thing the worker does: this object may be freed the moment it reads false, so nothing may follow it in execute() or loop().

Remarks

No rule of five, deliberately. An execution is declared once and used in place; the members already delete the copy operations several times over.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a std::shared_ptr, which is what binding to it requires.

An execution built any other way is still usable - `add_action()` takes actions directly - but cannot be bound, and the attempt does not compile.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A std::shared_ptr owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by untangle::bind()), and assigns to action a callable that passes it to [add_action\(\)](#). The execution is held weakly, so the action may outlive it.

Attention

Invoking action after the execution has been destroyed throws untangle::invalid_action. The worker catches it and drops the action with a warning.

Parameters

action	[in,out] - An action of type std::function<...>.
obj	- A std::shared_ptr that wraps the bound class object.
method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- A std::shared_ptr owning the execution the action is queued on. Requiring it here is what keeps an execution that no shared_ptr owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    const T & Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a plain function.

It wraps Fn in an action and assigns to action a callable that passes it to [add_action\(\)](#). The execution is held weakly, as in [bind_action_and_method\(\)](#).

Parameters

action	[in,out] - An action of type std::function<...>.
Fn	- A plain function.
async_exec	- A std::shared_ptr owning the execution the action is queued on.

8.2.3.4 `is_running()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Is this execution still working?

False from the moment the worker starts reporting itself finished, so a caller polling it learns a run is over without waiting for the object to become safe to destroy.

Remarks

[on_finished](#) is raised from inside the drain and reads this as true - the worker is standing in the callback with the rest of its path still to go.

Returns

true - the execution is still working.

8.2.3.5 `is_busy()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const [inline]
```

Is this execution working through its action list?

True from the moment an action is added until the last one has returned. Unlike [is_running\(\)](#), which is about the worker thread, this answers "has it room for more work"

- a worker started by [start\(\)](#) is running for its whole life whether busy or idle.

Remarks

Answers for this execution's own actions, not for those it triggers through [attach\(\)](#).

A false answer is durable only for the caller that is itself adding the actions; a true one may stop being true the moment after.

Returns

true - actions are queued, or one is running.

8.2.3.6 `run()`

```
template<typename actionT>
void untangle::async::execution< actionT >::run () [inline]
```

Runs what is queued on a worker of its own, once, and keeps the results.

The worker is detached: wait for it with [is_running\(\)](#) or [execution_poll](#). Actions queued while it is draining are run by it too. [results\(\)](#) is filled by this path only.

8.2.3.7 start()

```
template<typename actionT>
void untangle::async::execution< actionT >::start () [inline]
```

Starts a continuous worker that runs whatever is queued until `stop()`.

Unlike `run()`, the worker stays for the life of the execution, so `is_running()` is true whether or not there is work; ask `is_busy()` instead. Results are not collected.

8.2.3.8 stop()

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to `add_action()` after this call is refused, not queued: `stop()` is the end of this execution's working life, and an action accepted then would never run.

8.2.3.9 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_action (
    actionT action,
    Args... args) [inline]
```

Queues an action, bound to args, and says whether it was taken.

It runs later, on whichever thread drives this execution - its own worker, or an attacher's. Queueing before `run()` or `start()` is normal.

Attention

A stopped execution refuses and drops the action, silently for a caller that ignores the answer. Nothing is thrown: `untangle::invalid_action` means a dead binding, while a refused action is sound and merely too late.

Remarks

A caller arriving through `bind_action_and_method()` or `bind_action_and_function()` never sees the answer: those lambdas return `actionT::result_type`, which has no room for it.

Template Parameters

Args	- The argument types the action is bound to.
------	--

Parameters

action	- The action to queue.
args	- The arguments to bind to action.

Returns

- true - queued, and this execution's worker will run it.
- false - the execution is stopped and the action was dropped.

8.2.3.10 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too. Actions of different types can be run on one thread this way.

The attachment is a pointer into other; what makes that safe is the record left on the other side, which [~execution\(\)](#) and [detach\(\)](#) use to take it back out.

Parameters

other	- A std::shared_ptr owning the execution to trigger; requiring one keeps an unowned execution from being attached at all. Attaching the same execution twice adds it twice, and one detach() still removes it completely.
-------	---

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which keeps the graph a forest and the cycle check a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, itself included. Driving a cycle recurses until the stack is gone, so it is refused where the caller can still be told.

8.2.3.11 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other) [inline]
```

Detaches an execution previously attached by [attach\(\)](#).

Both paths are unwired together, the triggering one and the stopping one.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-------	---

Returns

true - other was attached and is no longer.
false - other was not attached to this execution.

8.2.3.12 results()

```
template<typename actionT>
auto untangle::async::execution< actionT >::results () const [inline]
```

The return values of the actions run by [run\(\)](#), in the order they ran.

Read it from [on_finished](#), which fires once the queue has drained, so the results are complete by the time the callback sees them. Returned by value: the worker is appending to it.

Remarks

Only [run\(\)](#) fills this. A continuous worker has no point at which a run is over, so [start\(\)](#) collects nothing.

Returns

The results of the most recent run, or empty if none has produced any.

8.2.4 Member Data Documentation

8.2.4.1 action_execute

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_execute
```

Connection point that runs this execution's queued actions. Bound to [execute_actions\(\)](#).

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.2 `action_stop`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_stop
```

Connection point that stops this execution. Bound to [stop\(\)](#).

[attach\(\)](#) wires it together with [action_execute](#), and [detach\(\)](#) unwires both.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.3 `action_is_running`

```
template<typename actionT>
std::function<bool(void)> untangle::async::execution< actionT >::action_is_running
```

Connection point that reports whether this execution is running. Bound to [is_running\(\)](#).

[execution_poll](#) holds its address, and [~execution\(\)](#) takes it back out.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.4 `on_finished`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::on_finished
```

Called on the worker's thread once the action list has drained. Assigned by the caller.

[results\(\)](#) is complete by the time it runs, and [is_running\(\)](#) reads true there.

The documentation for this class was generated from the following file:

- `async.hpp`

8.3 `untangle::async::execution_poll` Class Reference

Single-tone poll that reports whether any [execution](#) added to it is still running.

```
#include <async.hpp>
```

Public Member Functions

- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
 Adds an [execution](#) object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
 Removes an [execution](#) object from the poll.
- `auto is_running ()`
 Checks if the execution objects are running.

Static Public Member Functions

- `static execution_poll & get ()`
 Gets the single-tone instance.

8.3.1 Detailed Description

Single-tone poll that reports whether any [execution](#) added to it is still running.

Remarks

An execution runs in a detached thread and cannot be joined, so checking the "running" state is the only way to wait for one. This checks any number of them at once.

8.3.2 Member Function Documentation

8.3.2.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

<code>async_exec</code>	An execution object.
-------------------------	--------------------------------------

8.3.2.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

Generated by Doxygen

Attention

[add\(\)](#) stores a pointer into the object, so one destroyed while still added would leave the poll calling

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.2.3 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

8.3.2.4 get()

```
execution_poll & untangle::async::execution_poll::get () [inline], [static]
```

Gets the single-tone instance.

Returns

[execution_poll](#) An [execution_poll](#) instance.

The documentation for this class was generated from the following file:

- [async.hpp](#)

8.4 untangle::async::invalid_attachment Struct Reference

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

Generated by Doxygen

Remarks

Not an untangle::invalid_action: that one reports a binding whose target has died and is swallowed by the kernel while this one reports a binding whose target has died and is not swallowed

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 what()

```
const char * untangle::async::invalid_attachment::what () const    [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- async.hpp

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <chrono>
00012 #include <condition_variable>
00013 #include <exception>
00014 #include <functional>
00015 #include <list>
00016 #include <memory>
00017 #include <mutex>
00018 #include <print>
00019 #include <string>
00020 #include <thread>
00021 #include <type_traits>
00022 #include <utility>
00023 #include <vector>
00024
00025 namespace untangle {
00026 template <typename actionT>
00027 struct actuator;
00028
00029 namespace async {
00030 template <typename actionT>
00031 class execution;
00032 }
00033 } // namespace untangle
00034
00035 namespace untangle {
00036 namespace async {
00040
00047 struct invalid_attachment : std::exception {
00053     explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00054
00060     const char* what() const noexcept override { return message_.c_str(); }
00061
00062 private:
00063     std::string message_;
00064 };
00065
00074 struct attachment {
00076     std::weak_ptr<attachment> attacher;
00077 };
00078
00085 class execution_poll {
00086 public:
00092     template <typename asyncexecT>
00093     void add(asyncexecT& async_exec) {
00094         std::lock_guard<std::mutex> lock(actuator_mutex_);
00095
00096         if (!actuator_is_running_.is_connected()) {
00097             actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00098         } else {
00099             actuator_is_running_.add(&async_exec.action_is_running);

```

```

00100     }
00101 }
00102
00111 template <typename asyncexecT>
00112 void remove(asyncexecT& async_exec) {
00113     std::lock_guard<std::mutex> lock(actuator_mutex_);
00114
00115     actuator_is_running_.remove(&async_exec.action_is_running);
00116 }
00117
00123 auto is_running() {
00124     std::lock_guard<std::mutex> lock(actuator_mutex_);
00125     actuator_is_running_();
00126
00127     auto result = false;
00128     for (const auto& ret : actuator_is_running_.results) {
00129         result |= ret;
00130     }
00131     return result;
00132 }
00133
00139 static execution_poll& get() {
00140     static execution_poll instance;
00141     return instance;
00142 }
00143
00144 private:
00145     execution_poll() = default;
00146     ~execution_poll() = default;
00147     actuator<std::function<bool(void)>> actuator_is_running_;
00148     mutable std::mutex actuator_mutex_;
00149 };
00150
00164 template <typename actionT>
00165 class execution {
00166     // An execution may attach one of a different specialisation, which is a different class;
00167     // attach() and detach() reach into its attachment_lifetime_.
00168     template <typename otherActionT>
00169     friend class execution;
00170
00171 public:
00181     template <typename... Args>
00182     static std::shared_ptr<execution> create_instance(Args&&... args) {
00183         return std::make_shared<execution>(std::forward<Args>(args)...);
00184     }
00185
00191     execution() : execution(std::string(default_name)) {}
00192
00198     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00199         other_this_ = this;
00200         action_execute_ = untangle::bind(other_this_, &execution<actionT>::execute_actions);
00201         action_stop_ = untangle::bind(other_this_, &execution<actionT>::stop);
00202         action_is_running_ = untangle::bind(other_this_, &execution<actionT>::is_running);
00203     }
00204
00219     ~execution() {
00220         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00221         // may reach this object, and the actions about to be destroyed are the ones it holds.
00222         if (!attachment_lifetime_->attacher.expired()) {
00223             attacher_execute_->remove(&action_execute_);
00224             attacher_stop_->remove(&action_stop_);
00225         }
00226
00227         {
00228             std::lock_guard<std::mutex> lock(action_mutex_);
00229             started_ = false;
00230             stopped_ = true;
00231         }
00232
00233         action_cv_.notify_all();
00234
00235         while (running_.load()) { // time of check
00236             std::this_thread::sleep_for(std::chrono::milliseconds(1));
00237         }
00238
00239         // time of use: this object is freed once the destructor returns, so the check above is only
00240         // safe because running_ is the last thing the worker touches
00241         execution_poll::get().remove(*this);
00242     }
00243
00261     template <typename classT, typename T>
00262     static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00263                                       T classT::* method,
00264                                       const std::shared_ptr<execution>& async_exec) {
00265         actionT async_action = untangle::bind(obj, method);
00266
00267         action = [wp = std::weak_ptr<execution>(async_exec),

```

```

00268         async_action](auto... args) -> actionT::result_type {
00269     // lock() also keeps the execution alive for the duration of the call
00270     const auto exec = wp.lock();
00271     if (!exec) {
00272         throw invalid_action("bind: invalid execution");
00273     }
00274     exec->add_action(async_action, args...);
00275     return typename actionT::result_type();
00276 };
00277 }
00278
00289 template <typename T>
00290 static void bind_action_and_function(actionT& action, const T& Fn,
00291                                     const std::shared_ptr<execution>& async_exec) {
00292     actionT async_action = Fn;
00293
00294     action = [wp = std::weak_ptr<execution>(async_exec),
00295             async_action](auto... args) -> actionT::result_type {
00296         const auto exec = wp.lock();
00297         if (!exec) {
00298             throw invalid_action("bind: invalid execution");
00299         }
00300         exec->add_action(async_action, args...);
00301         return typename actionT::result_type();
00302     };
00303 }
00304
00316 bool is_running() const { return running_.load() && !finishing_.load(); }
00317
00333 bool is_busy() const {
00334     std::lock_guard<std::mutex> lock(action_mutex_);
00335     return !action_list_.empty() || executing_action_.load();
00336 }
00337
00344 void run() {
00345     running_ = true;
00346     finishing_ = false;
00347     collecting_results_ = true;
00348     this_thread_ = std::thread(&execution::execute, this);
00349     this_thread_.detach();
00350 }
00351
00358 void start() {
00359     running_ = true;
00360     finishing_ = false;
00361     collecting_results_ = false;
00362     {
00363         std::lock_guard<std::mutex> lock(action_mutex_);
00364         started_ = true;
00365         stopped_ = false;
00366     }
00367
00368     this_thread_ = std::thread(&execution::loop, this);
00369     this_thread_.detach();
00370 }
00371
00378 void stop() {
00379     {
00380         std::lock_guard<std::mutex> lock(action_mutex_);
00381         started_ = false;
00382         stopped_ = true;
00383     }
00384
00385     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00386     // spin on the list; waking it is enough.
00387     action_cv_.notify_all();
00388
00389     if (actuator_stop_.is_connected()) {
00390         actuator_stop_();
00391     }
00392 }
00393
00415 template <typename... Args>
00416 bool add_action(actionT action, Args... args) {
00417     {
00418         std::lock_guard<std::mutex> lock(action_mutex_);
00419
00420         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00421         // here is what keeps it from sitting in the list looking as though it were pending.
00422         if (stopped_) {
00423             std::println(stderr, "warning: execution '{}{}' is stopped, action not added", name);
00424             return false;
00425         }
00426
00427         action_list_.push_back(std::bind(action, args...));
00428     }
00429 }

```

```

00430     action_cv_.notify_one();
00431     return true;
00432 }
00433
00453 template <typename otherT>
00454 void attach(const std::shared_ptr<otherT>& other) {
00455     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00456         throw invalid_attachment("attach: an execution cannot be attached to itself");
00457     }
00458
00459     if (!other->attachment_lifetime_->attacher.expired()) {
00460         throw invalid_attachment("attach: that execution is attached already");
00461     }
00462
00463     // At most one attacher each makes the graph a forest, so a cycle closes exactly when other is
00464     // somewhere up this chain. Walking it is the whole check.
00465     for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00466         if (link == other->attachment_lifetime_) {
00467             throw invalid_attachment("attach: the attachment would close a cycle");
00468         }
00469     }
00470
00471     if (!actuator_execute_.is_connected()) {
00472         actuator_execute_ = untangle::connect(other->action_execute);
00473     } else {
00474         actuator_execute_.add(&other->action_execute);
00475     }
00476
00477     if (!actuator_stop_.is_connected()) {
00478         actuator_stop_ = untangle::connect(other->action_stop);
00479     } else {
00480         actuator_stop_.add(&other->action_stop);
00481     }
00482
00483     // What ~execution() needs to take itself back out of these actuators: the link proves this
00484     // attacher is still there, and the two pointers are only ever followed while it does.
00485     other->attachment_lifetime_->attacher = attachment_lifetime_;
00486     other->attacher_execute_ = &actuator_execute_;
00487     other->attacher_stop_ = &actuator_stop_;
00488 }
00489
00501 template <typename otherT>
00502 bool detach(otherT& other) {
00503     // The actuator's own action list is the record of what is attached; nothing else has to keep
00504     // one. An action is stored as a pointer, so the attachment is found by identity.
00505     const auto& actions = actuator_execute_.actions;
00506     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00507         return false;
00508     }
00509
00510     actuator_execute_.remove(&other.action_execute);
00511     actuator_stop_.remove(&other.action_stop);
00512
00513     other.attachment_lifetime_->attacher.reset();
00514     other.attacher_execute_ = nullptr;
00515     other.attacher_stop_ = nullptr;
00516
00517     return true;
00518 }
00519
00531 auto results() const {
00532     std::lock_guard<std::mutex> lock(results_mutex_);
00533     return results_;
00534 }
00535
00536 static constexpr auto default_name = "default_name";
00537
00543 std::function<void(void)> action_execute;
00544
00552 std::function<void(void)> action_stop;
00553
00562 std::function<bool(void)> action_is_running;
00563
00569 std::function<void(void)> on_finished;
00570 std::string name = default_name;
00571
00572 private:
00573     using queued_action_t = std::function<typename actionT::result_type(void)>;
00574
00581 void execute_action(queued_action_t& action) {
00582     if constexpr (std::is_void_v<typename actionT::result_type>) {
00583         action();
00584     } else {
00585         auto value = action();
00586
00587         if (collecting_results_) {
00588             std::lock_guard<std::mutex> lock(results_mutex_);

```

```

00589     results_.push_back(std::move(value));
00590 }
00591 }
00592 }
00593
00603 void execute_actions() {
00604     actions_run_ = 0;
00605
00606     // The action is taken off the list under the lock and invoked with the lock released: an action
00607     // is caller code that may run for a while, and may itself call add_action().
00608     for (;;) {
00609         queued_action_t action;
00610
00611         {
00612             std::lock_guard<std::mutex> lock(action_mutex_);
00613             if (action_list_.empty()) {
00614                 break;
00615             }
00616
00617             action = std::move(action_list_.front());
00618             action_list_.pop_front();
00619
00620             // Under the lock that emptied the list, so is_busy() sees the pop and this together or
00621             // neither.
00622             executing_action_ = true;
00623         }
00624
00625         // Nothing may leave this loop: an exception escaping a detached thread function calls
00626         // std::terminate.
00627         try {
00628             execute_action(action);
00629         } catch (const invalid_action& ia) {
00630             std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00631                 ia.what());
00632         } catch (const std::exception& e) {
00633             std::println(stderr, "warning: execution '{}' dropped an action that threw: {}", name,
00634                 e.what());
00635         } catch (...) {
00636             std::println(stderr, "warning: execution '{}' dropped an action that threw an unknown type",
00637                 name);
00638         }
00639
00640         executing_action_ = false;
00641
00642         // Counted whether it ran cleanly, reported a dead binding, or threw: it came off the queue,
00643         // and the queue is what the notification is about.
00644         ++actions_run_;
00645
00646         // The notification is raised without the lock: on_finished may call add_action(), which
00647         // takes it.
00648         bool drained = false;
00649         {
00650             std::lock_guard<std::mutex> lock(action_mutex_);
00651             drained = action_list_.empty();
00652         }
00653
00654         if (drained) {
00655             notify_finished();
00656         }
00657     }
00658
00659     // Last, and part of the pass: an attachment is only ever reached through action_execute, which
00660     // is this function, so whoever drives this execution drives the ones attached to it.
00661     if (actuator_execute_.is_connected()) {
00662         actuator_execute_();
00663     }
00664 }
00665
00670 void notify_finished() {
00671     if (actions_run_ == 0 || !on_finished) {
00672         return;
00673     }
00674
00675     {
00676         // An action may queue another, so a pass that drained can leave more behind it. That is the
00677         // next batch, not the end of this one.
00678         std::lock_guard<std::mutex> lock(action_mutex_);
00679         if (!action_list_.empty()) {
00680             return;
00681         }
00682     }
00683
00684     on_finished();
00685 }
00686
00687 void execute() {
00688     // The results belong to this run, not to this run and the last one.

```

```

00689     {
00690         std::lock_guard<std::mutex> lock(results_mutex_);
00691         results_.clear();
00692     }
00693
00694     execute_actions();
00695
00696     // After the drain and the callback it raised: there is no more work to be told about.
00697     finishing_ = true;
00698
00699     std::println("execution '{}{}' finishing thread", name);
00700
00701     // Must stay last: ~execution() may free this object the moment it reads false.
00702     running_ = false;
00703 }
00704
00705 void loop() {
00706     for (;;) {
00707         {
00708             std::unique_lock<std::mutex> lock(action_mutex_);
00709
00710             // Bounded: an attached execution has its own list and cannot notify this condition
00711             // variable.
00712             action_cv_.wait_for(lock, std::chrono::milliseconds(10),
00713                 [this] { return !action_list_.empty() || !started_; });
00714
00715             if (!started_ && action_list_.empty()) {
00716                 break;
00717             }
00718         }
00719         execute_actions();
00720     }
00721
00722     // A last pass for the attachments; this execution's own list is already empty here.
00723     execute_actions();
00724
00725     // After the last pass, because that pass still runs work.
00726     finishing_ = true;
00727
00728     std::println("execution '{}{}' thread finished", name);
00729
00730     // Must stay last: ~execution() may free this object the moment it reads false.
00731     running_ = false;
00732 }
00733
00734 std::thread this_thread_;
00735
00736 std::atomic_bool started_ = {false};
00737 std::atomic_bool running_ = {false};
00738
00739 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00740 // may be queued before run()/start(), but not after stop().
00741 std::atomic_bool stopped_ = {false};
00742
00743 // action_list_, started_ and stopped_ are written by every thread that calls add_action() or
00744 // stop() and read by the worker; nothing touches them outside this mutex.
00745 mutable std::mutex action_mutex_;
00746 std::condition_variable action_cv_;
00747
00748 std::list<std::function<typename actionT::result_type(void)>> action_list_;
00749
00750 using void_actuator = actuator<std::function<void(void)>>;
00751
00752 void_actuator actuator_execute_;
00753 void_actuator actuator_stop_;
00754
00755 std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
00756
00757 void_actuator* attacher_execute_ = nullptr;
00758 void_actuator* attacher_stop_ = nullptr;
00759
00760 // std::vector cannot hold void type; use an arbitrary type e.g. int
00761 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00762     typename actionT::result_type>;
00763
00764 std::vector<typename resultT::type> results_;
00765
00766 mutable std::mutex results_mutex_;
00767
00768 std::atomic_bool collecting_results_ = {false};
00769
00770 std::atomic_bool finishing_ = {false};
00771
00772 std::atomic_bool executing_action_ = {false};
00773
00774 std::atomic_size_t actions_run_ = {0};
00775

```

```
00841  execution* other_this_;  
00842 };  
00843 } // namespace async  
00844 } // namespace untangle
```


Index

- ~execution
 - untangle::async::execution< actionT >, [20](#)
- action_execute
 - untangle::async::execution< actionT >, [25](#)
- action_is_running
 - untangle::async::execution< actionT >, [26](#)
- action_stop
 - untangle::async::execution< actionT >, [25](#)
- add
 - untangle::async::execution_poll, [27](#)
- add_action
 - untangle::async::execution< actionT >, [23](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [24](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [21](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [20](#)
- create_instance
 - untangle::async::execution< actionT >, [20](#)
- detach
 - untangle::async::execution< actionT >, [24](#)
- execution
 - untangle::async::execution< actionT >, [19](#)
- get
 - untangle::async::execution_poll, [28](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [28](#)
- is_busy
 - untangle::async::execution< actionT >, [22](#)
- is_running
 - untangle::async::execution< actionT >, [21](#)
 - untangle::async::execution_poll, [28](#)
- namespace untangle: functions, [13](#)
- on_finished
 - untangle::async::execution< actionT >, [26](#)
- remove
 - untangle::async::execution_poll, [27](#)
- results
 - untangle::async::execution< actionT >, [25](#)
- run
 - untangle::async::execution< actionT >, [22](#)
- start
 - untangle::async::execution< actionT >, [22](#)
- stop
 - untangle::async::execution< actionT >, [23](#)
- untangle, [15](#)
- untangle::async::attachment, [17](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [20](#)
 - action_execute, [25](#)
 - action_is_running, [26](#)
 - action_stop, [25](#)
 - add_action, [23](#)
 - attach, [24](#)
 - bind_action_and_function, [21](#)
 - bind_action_and_method, [20](#)
 - create_instance, [20](#)
 - detach, [24](#)
 - execution, [19](#)
 - is_busy, [22](#)
 - is_running, [21](#)
 - on_finished, [26](#)
 - results, [25](#)
 - run, [22](#)
 - start, [22](#)
 - stop, [23](#)
- untangle::async::execution_poll, [26](#)
 - add, [27](#)
 - get, [28](#)
 - is_running, [28](#)
 - remove, [27](#)
- untangle::async::invalid_attachment, [28](#)
 - invalid_attachment, [28](#)
 - what, [29](#)
- what
 - untangle::async::invalid_attachment, [29](#)